

VaultMind

个人知识库 RAG 问答与评测系统 面试答辩手册

181 篇笔记 / 33.1 万字 / 516 条双链 → 可问答 · 可评测 · 可复现
零 LangChain · 零 torch · 零向量数据库

指标: $\text{Recall@5} = 0.9333$ · $\text{MRR} = 0.8672$ · $\text{nDCG@10} = 0.8840$ (三项达标)
git 2d71a90 · 2026-09 · 面向 AI 应用开发 + AI 数据分析双方向求职

1 · 指标总览 (60 条 gold 评测集 · bm25 检索 · 全部可复现)

指标	基线值	目标	判定	出处
Recall@1	0.8167	—	—	reports/baseline.md
Recall@5	0.9333	≥ 0.80	✓ 达标	reports/baseline.md
Recall@10	0.9333	—	—	reports/baseline.md
MRR	0.8672	≥ 0.65	✓ 达标	reports/baseline.md
nDCG@10	0.8840	≥ 0.70	✓ 达标	reports/baseline.md
平均延迟	0.003 s	—	—	reports/baseline.md
引用可追溯	非法引用 = 0	100%	✓	reports/m4_smoke.md
超纲拒答	2/2 探针	拒答	✓	reports/m4_smoke.md
消融实验	6 组 × 60 gold	基线闸门	✓	reports/ablation.md

复现: `python -m vaultmind.eval` (基线) | `python scripts\m6_ablation.py` (消融) | `python scripts\m6b_chunk_ablation.py` (分块粒度消融)

2 · 系统架构 (四层管线)

- ① 数据管道: Obsidian 源库(只读) → 扫描/体检 → 结构感知分块 (H2 单元、超 600 字二切、
[文档|章节|标签] 前缀注入) → SQLite(docs=181/chunks=1437/links=516) + FTS5(jieba)
- ② 检索层: BM25(FTS5) + bge-m3 向量(numpy 暴力 cosine) → RRF 融合(k=60)
- ③ 生成层: [S#] 编号上下文(≤2块/文档, 6000字符预算) → qwen2.5:7b 本地生成
→ 引用校验(编号∈1..n) + 超纲拒答(话术规则双层)
- ④ 产品层: FastAPI(/ask /search /stats /metrics /badcases /feedback) + 零依赖 Web
问答页/看板; 引用 obsidian:// 跳回原笔记; qa_logs 四分类归因

核心设计决策

- 结构感知分块: 一块三用 (引用溯源 / 语义上下文 / 评测可追溯)
- BM25 检索 (默认): 消融验证单路 R@5 0.9333 反超融合; RRF k 参数不敏感
- 评测集防自欺: 候选不用 LLM 生成, 分层抽样定稿, seed=42 重跑逐字节一致
- 引用 100% 可追溯: 提示词约束 + 规则校验双层 (4 种格式变异探针全覆盖)

3 · 六组消融（每个组件增益量化，负结果如实记录）

实验	关键结果	结论
三路单拆	B/V/H = 0.9333 / 0.8333 / 0.9167; R@1 融合 0.7667 < 向量 0.7000	融合 R@5 正增益 -0.0166; 量化「硬部稀疏」
RRF k=20/60/100	R@5 恒 0.9167, MRR 极差 0.0063	参数不敏感, 无需调参
查询改写	规则 +0.017 / LLM -0.017; hard 档一路降	负结果: 不上线
标题加权重排	easy 1.0000 / hard 0.7917	分层撕裂: 需条件化
双链 1-hop 扩展	R@5 +0.0000	中性: 转「相关笔记推荐」
规模-延迟曲线	25%→100% 语料 235→221 ms; 点积 <1ms	零向量库决策被验证

坏例账本: 11 题未进 Top-5 (10 题裸标题型) ; easy 档 100% / hard 档 79% → 分层归因。

3b · 分块粒度消融 (M6b: 查出「标题被剥离 + 前缀没进检索」)

变体 (分块策略)	规模	R@5	MRR	nDCG@10
V1 基线 h2/600	1321 块 / 190 字	0.8167	0.7099	0.7406
V2 细粒度 h3/300	1703 块 / 146 字	0.7833	0.6931	0.7201
V3 粗粒度 整篇/1500	262 块 / 1016 字	0.8500	0.7656	0.7983
V4 无前缀 (对照)	1321 块 / 190 字	0.8167	0.7099	0.7406
V5 前缀入检索	1321 块 / 276 字	0.9167	0.8311	0.8529
V6 组合 粗+前缀	262 块 / 1093 字	0.8333	0.8132	0.8261

结论: 细粒度 0.7833 (-0.0334) | 粗粒度 0.8500 (+0.0333) | 前缀入检索 0.9167 (+0.1000) | 组合 0.8333 不叠加 → 最优是 V5「前缀入检索」。

根因 (本轮最重要产出): 切块拿「## 标题」当分隔符, 标题文本被剥离出正文; 而标题只写进不参与 FTS/向量检索的 prefix 字段 → 裸标题查询在正文里没有任何字面锚点。这正是上一轮「查询改写」「标题重排」都救不回来的真因: 问题不在查询侧, 而在索引侧丢了标题。

铁证: V1 (前缀进字段) 与 V4 (无前缀) 的「检索文本指纹」完全相同 → 前缀此前对检索零贡献。V5 已采纳: prefix_mode=inline 为正式默认, 基线 R@5 0.9333。

4 · 高频面试问题与答法（节选，全文见 docs/Q&A预案.md）

Q 为什么不用 LangChain / 向量库 / torch?

A 数据规模决定架构：1,437 chunks × 1024 维点积实测 <1ms，向量库是过度设计；自写管道每个环节可解释、可改动；面试能讲清原理。

Q 评测集可信吗？

A 三道防线：候选只用笔记结构模板化派生（不用 LLM 生成问题）；分层抽样定稿 seed=42 可复现 + SHA-256；每条 gold 目标必须映射回已索引 chunk（实测 100%）。

Q 引用可追溯怎么保证？LLM 编造怎么办？

A 提示词约束 + 规则校验双层：引用编号必须落在上下文 1..n；实测模型有 4 种格式变异（列表式等），解析器宽容 + 探针全覆盖；超纲 2/2 拒答。

Q 坏例怎么处理？

A 先分层归因（easy 100% / hard 79%，主因=裸标题查询），再消融验证方案：改写与 1-hop 被数据否决，标题重排需条件化——不是所有坏例都要修，先算账。

Q 最难的三个坑？

A 评测集防自欺、LLM 输出格式不稳定、Windows 环境工程（全路径/编码/服务掉线/bat 端口秒退）——全部沉淀成知识卡片。

5 · 数字速查表 (每个数字可指路出处)

数字	含义	出处
181 / 33.1万字 / 1067 H2 / 516 双链	知识库体检基线	audit_report.md
1,437 chunks / 管道 1.2s	索引规模与速度	M1
0.9333 / 0.8672 / 0.8840	60 条 gold 官方基线 (三项达标)	baseline.md
11 题未进 Top-5 (100% / 79%)	坏例分层	baseline.md
非法引用 0 · 拒答 2/2 · 生成 8~15s	生成链路	m4_smoke.md
-0.0166 / +0.017 / -0.017 / +0.0000	消融增益与负结果	ablation.md
分块粒度消融 -0.0334 +0.0333 +0.1000	M6b: 细/粗/前缀入检索 (候选)	m6b_chunk_ablation.md
点积 <1ms · embedding ~280ms	规模曲线 → 零向量库	ablation.md E6
pytest 84 条 · pre-commit 钩子	三层防线	tests/

复现与演示

- 环境体检: D:\python\python.exe scripts\env_check.py (13 项 PASS)
- Web 演示: run_api.bat → http://127.0.0.1:8001 (问答页 + 分析看板)
- 问答 CLI: D:\python\python.exe -m vaultmind.ask "问题"
- 全量测试: D:\python\python.exe -m pytest tests -q